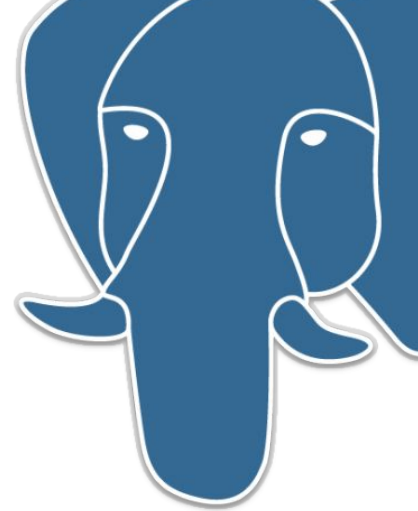


PostgreSQL Access Policy

practical tips



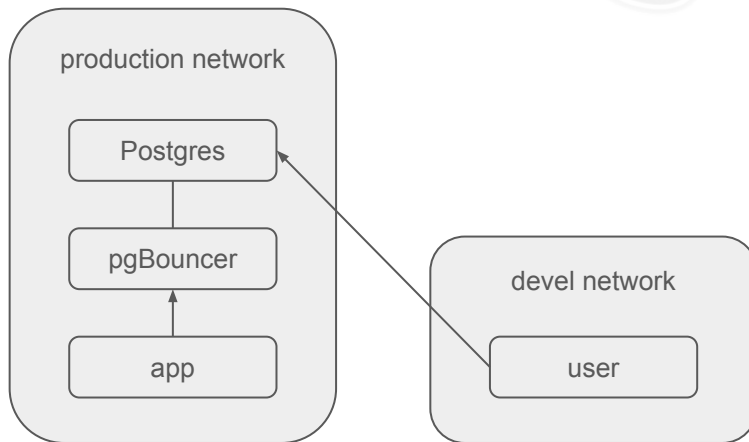
Why do we do that

- **Security** - control who can access what
- **Data safety** - prevent unauthorized or accidental changes
- **Auditing** - establish and demonstrate who can access or modify data
- **Monitoring & performance** - make access patterns easier to monitor and optimize
- **A clean API** - expose only the objects and operations that applications actually need



Access Layers

- **Data Network / VPN**
 - restrict where connections can originate
- **Proxy (pgBouncer)**
 - e.g. pgBouncer
 - connection limits and pooling
 - optionally separate connection endpoints/policies
- **Postgres' pg_hba.conf**
 - controls which clients, databases and roles may connect
 - controls the authentication method
 - can restrict access by IP address/subnet
- **Postgres Authentication**
 - user/password
 - GSAPI/Kerberos
 - Client certificates



Access Layers

- **Data access privileges**
 - database
 - schemas
 - tables, views
 - columns
 - rows (RLS)
 - functions, procedures
- **Others privileges**
 - database and schema/object creation
 - publications and subscriptions
 - extensions
 - objects within schemas
 - temporary tables
 - triggers and foreign keys
 - session/server configuration parameters
 - maintenance and administrative operations



Roles

- Key catalog tables/views:
 - pg_authid (t) -> pg_roles (v)
 - pg_auth_members (membership / inheritance)
 - pg_db_role_setting - settings
 - pg_shdescription - comments
- Non-inheritable properties:
 - attributes: SUPERUSER, CREATEROLE, CREATEDB, CONNECTIONS, LOGIN, PASSWORD, PWD VALID UNTIL INHERIT, REPLICATION, BYPASSRLS
 - settings, ie ALTER ROLE SET log_statement = 'all';
- Role groups: roles with NOLOGIN
- CREATE ROLE, CREATE USER, CREATE GROUP



The 'public' pseudo-role

- group that contains all roles
- hardcoded - not in catalog, but **oid=0**
- lowercase - **do not use information_schema privileges !!!**
- access granted by default on:
 - database: CONNECT, TEMPORARY,
 - public schema: USAGE
 - functions/procedures: EXECUTE
 - languages: USAGE
 - before pg15
 - public schema: CREATE
 - retains with update

```
FOR rcd IN
    SELECT *
    FROM information_schema.table_privileges;
LOOP
    EXECUTE format('GRANT ... TO %I', grantee);
END LOOP;
```

Fails on public pseudo-role

```
CREATE ROLE "PUBLIC";
GRANT SELECT ON TABLE mytable TO public, "PUBLIC";
```

```
SELECT table_name, privilege_type, grantee
FROM information_schema.table_privileges;
```

table_name	privilege_type	grantee
mytable	SELECT	PUBLIC
mytable	SELECT	PUBLIC

public and PUBLIC impossible to distinguish



The aclexplode()

acl format example: '{michal=arwdDxtm/michal,=r/michal}'

table_name	acl column	owner column
pg_attribute	attacl	attowner
pg_class	relacl	relowner
pg_database	datacl	datowner
pg_default_acl	defaclobjtype	defownerobjtype
pg_default_acl	defaclacl	defownerowner
pg_default_acl	defaclrole	defownerrole
pg_default_acl	defaclnamespace	defownernamespace
pg_foreign_data_wrapper	fdwaccl	fdwowner
pg_foreign_server	srvaccl	srvowner
pg_language	lanaccl	lanowner
pg_largeobject_metadata	lomaccl	lomowner
pg_namespace	nspacl	nspowner
pg_parameter_acl	paraccl	parowner
pg_proc	proaccl	proowner
pg_tablespace	spcaccl	spcowner
pg_type	typaccl	typowner

```
SELECT
    pc.oid::regclass      AS object,
    pc.relowner::regrole AS owner
    CASE x.grantee
        WHEN 0 THEN 'public'
        ELSE oid::regrole
    END,
    x.privilege_type
FROM pg_class AS pc
LEFT JOIN [LATERAL] aclexplode(relacl) AS x ON TRUE
```

The returned oid value for public role is 0 (zero)



Object identifier types

1. object identifier: oid
2. ::regrole, ::regclass, etc → fully qualified identifiers, **strict**, **quoted**
3. to_regrole(), to_regclass(), etc → NULL if not found
4. watch out the search_path!!!

```
SELECT
    member::regrole,
    roleid::regrole
FROM pg_catalog.pg_auth_members
---
pg_monitor    pg_read_all_settings
pg_monitor    pg_read_all_stats
...
```

```
SELECT
    oid::regclass,
    relnamespace::regnamespace
FROM pg_class
WHERE relnamespace = 'foo'::regnamespace      -- strict
    -- relnamespace::regnamespace::TEXT = 'foo'  -- tolerant
---
foo.bar    foo
```

```
SET search_path = 'foo';
SELECT oid::regprocedure
FROM pg_proc (...)
---
bar(integer)
```

```
SET search_path = '';
SELECT oid::regprocedure
FROM pg_proc (...)
---
foo.bar(integer)
```

missing schema



Privileges

- databases: CONNECT, TEMPORARY, CREATE
- tables: SELECT, INSERT, UPDATE, TRUNCATE
- sequences: USAGE, UPDATE
- functions: EXECUTE
- schemas: USAGE, CREATE
- maintenance: MAINTAIN, SET, ALTER SYSTEM

```
GRANT ... ON object_type object(s) TO role(s);  
REVOKE ... ON object_type object(s) FROM role(s);
```

Privileges are subject of inheritance

Testing privileges:

- has_*_privilege() - resolves inheritance, does not resolve object tree!!!



Inheriting privileges

1. Role's INHERIT/NOINHERIT attribute ← the default
2. Grant WITH
 - INHERIT <TRUE|FALSE> - overrides the default
 - SET <TRUE|FALSE> - controls switching to parent role

```
CREATE ROLE p;  
CREATE ROLE c [INHERIT];  
  
GRANT p TO c WITH INHERIT FALSE SET TRUE;  
  
GRANT SELECT ON TABLE test TO p;
```

```
-- logged as c  
SELECT * FROM test; -- permission denied
```

```
-- logged as c  
SET ROLE p;  
SELECT * FROM test; -- granted
```



Predefined privileges

1. Avoids sharing superuser and excessive privileges
2. Follows the principle of least privilege
3. Reduces custom role management and simplifies administration
4. Makes privileges easier to understand and audit
5. Standardizes access control across databases

Role	Introduced in
-----	-----
pg_checkpoint	PostgreSQL 15
pg_create_subscription	PostgreSQL 16
pg_database_owner	PostgreSQL 14
pg_execute_server_program	PostgreSQL 11
pg_maintain	PostgreSQL 17
pg_monitor	PostgreSQL 10
pg_read_all_data	PostgreSQL 14
pg_read_all_settings	PostgreSQL 10
pg_read_all_stats	PostgreSQL 10
pg_read_server_files	PostgreSQL 11
pg_signal_autovacuum_worker	PostgreSQL 18
pg_signal_backend	PostgreSQL 9.6
pg_stat_scan_tables	PostgreSQL 10
pg_use_reserved_connections	PostgreSQL 16
pg_write_all_data	PostgreSQL 14
pg_write_server_files	PostgreSQL 11

Default privileges

Confusing!!!

- set to objects added in future
- provided to a “target” role (public excluded)
- to set up privileges to another role (public included)

```
ALTER DEFAULT PRIVILEGES FOR ROLE target_role  
GRANT SELECT ON TABLE mytable TO role;
```

Applies the privilege to the role only if a new table is being created by target_role.



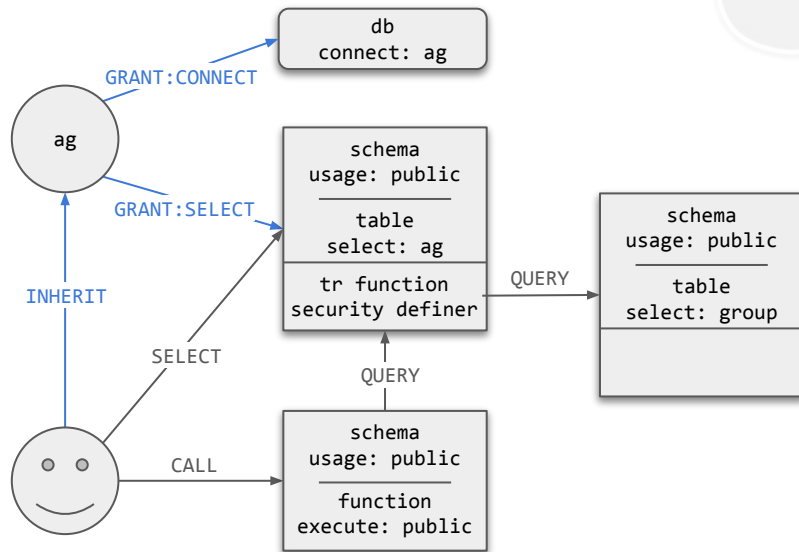
Interconnections

- dblink, FDW, logical replications
- problems:
 - credentials exposed in source code and/or system tables
 - (static) role mapping
- ways to avoid storing password
 - `use_scram_passthrough` since pg18
 - GSSAPI/Kerberos credential delegation since pg16
 - Client certificates
 - service files
 - dblink only
 - security definer function with superuser owner
 - set remote side role using `session_user`
 - `dblink_connect_u()` !

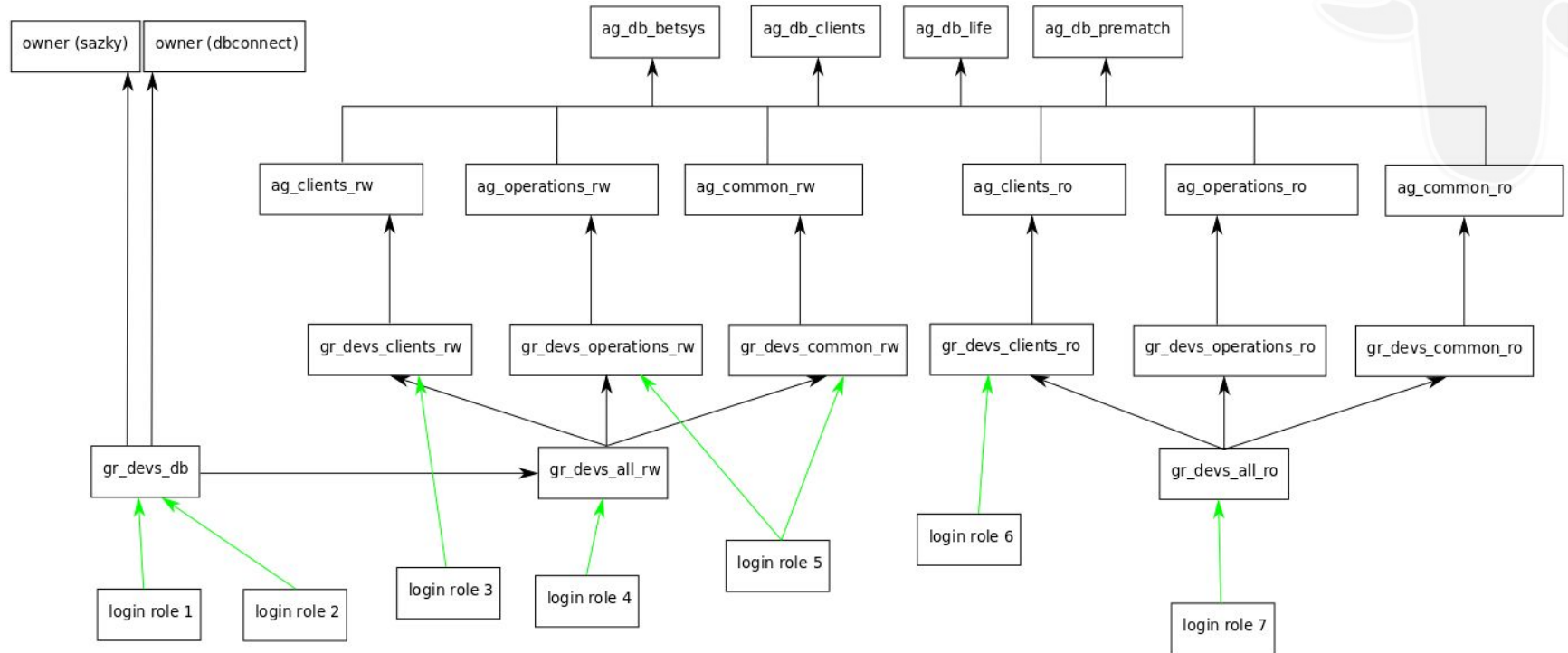


Approaches: table privileges based

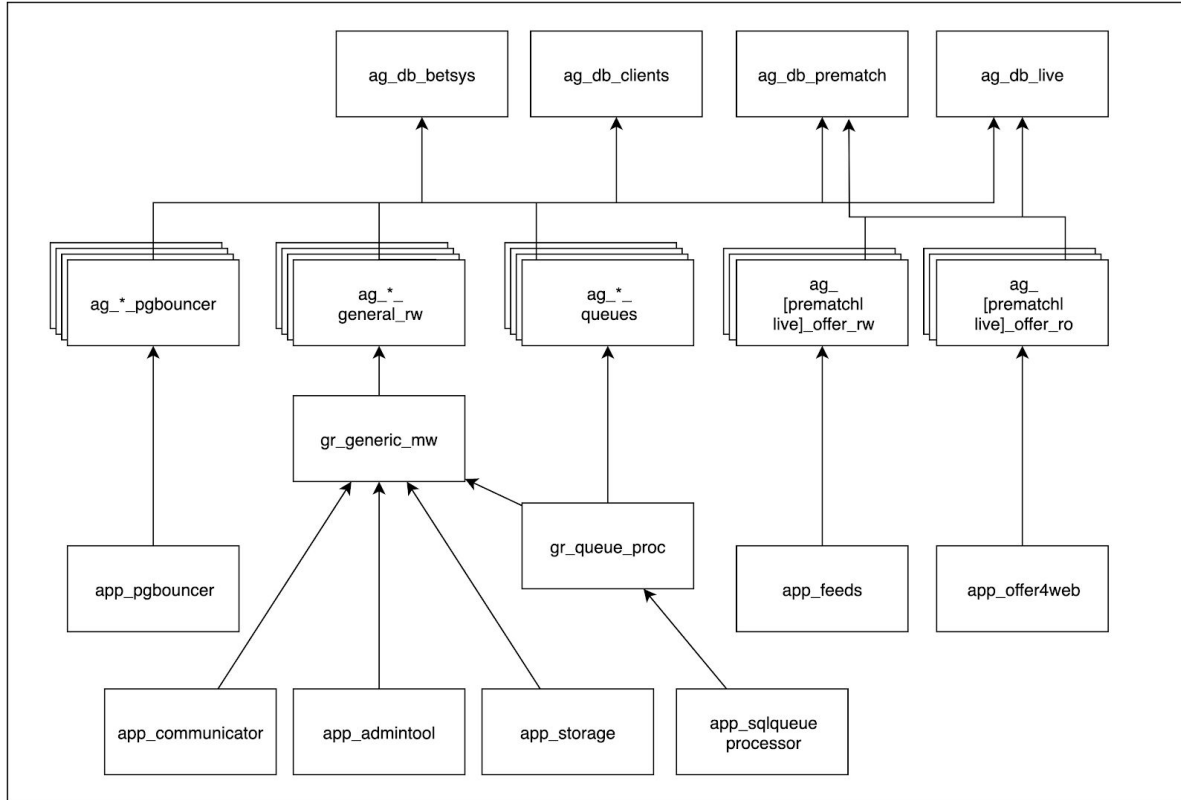
- for users
- for apps
 - with wide/unspecified data scope
 - that query tables directly
- functions and trigger functions: `execute public`
 - there are exceptions
 - trigger functions:
 - owned by tables owner
 - security definer
- schemas: `usage public`
- tables granted to access groups
- roles inherits from (access) groups



Approaches: table privileges based

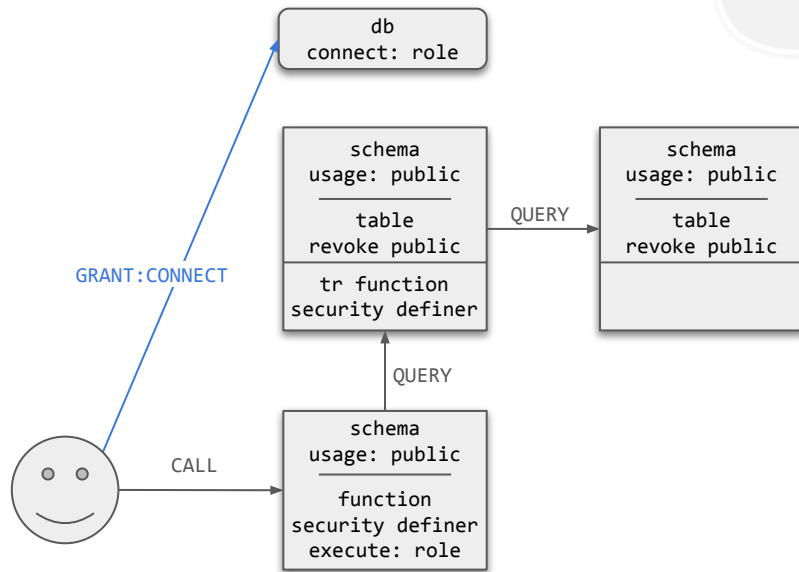


Approaches: table privileges based



Approaches: API function based

- access data **exclusively** through functions/procedures
- tables: no public access
- functions:
 - public access revoked,
 - granted to specific users instead
 - security definer
 - owner owns or has access to data



Approaches: Pros & Cons

Data access based

- + lowest impact during initial implementation
- + data remains accessible for users
- + pre-development access
- + preliminary/ad-hoc development
- become convoluted if function access
more tables are called by more apps

Functions access based

- + easier per-app management
- no direct access to data
- security definer
- dependency on functions development
- + minimal app-to-app, team-to-team influence



Conventions

Convention applied by us

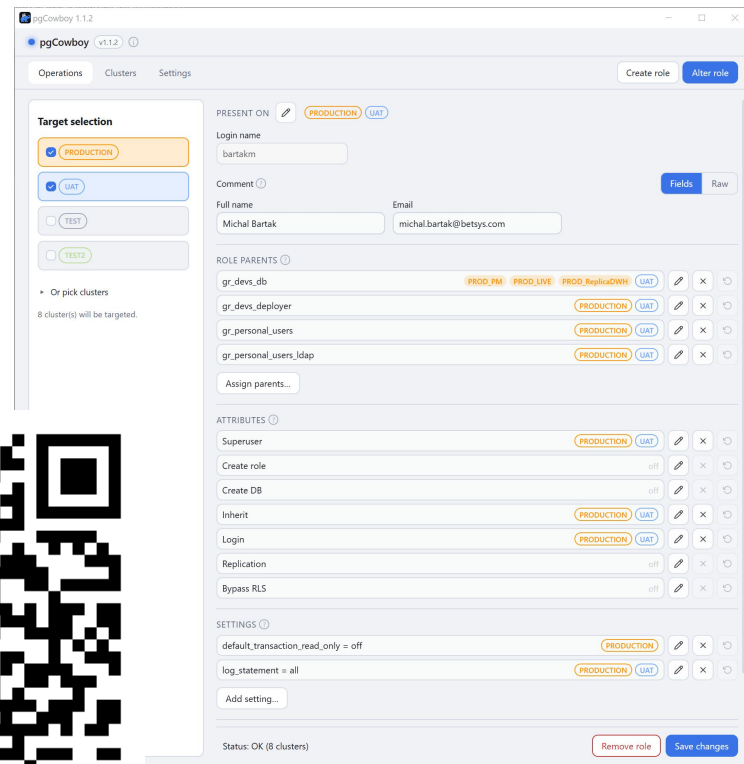
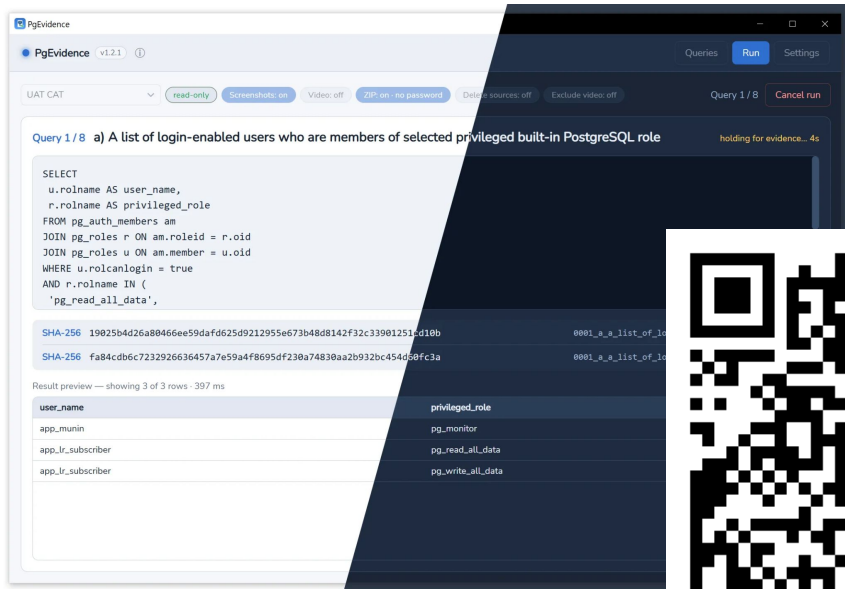
- user role follows company pattern
- app_* for application roles
- ag_* access groups - granted to objects (mandatory)
- gr_* regular groups - for grouping (optional)
- dedicated roles/groups
 - owner role (non-superuser)
 - deployment (superuser)
 - logical replications (subscription owner & remote connection)
 - owner for dblink connections
 - to distinguish a human role from an app role
- Never grant login roles direct access to objects such as databases, schemas, tables, or views.



Q&A



Thank you



<https://michal-bartak.github.io/postgresql-access-policy-landing-page/>